

Evolution & Improvement

The agency does not merely execute work. It learns from it.

Every completed task generates a signal—a scored evaluation measuring how well the agent performed against the task’s requirements and the agent’s own declared standards. These signals accumulate into performance records on agents, roles, and motivations. When enough data exists, an evolution cycle reads the aggregate picture and proposes structural changes: sharpen a role’s description, tighten a motivation’s constraints, combine two high-performers into something new, retire what consistently underperforms. The changed entities receive new content-hash IDs, linked to their parents by lineage metadata. Better identities produce better work. Better work produces sharper evaluations. The loop closes.

This is the autopoietic core of the agency system—a structured feedback loop where work produces the data that drives its own improvement.

Evaluation

Evaluation is the act of scoring a completed task. It answers a concrete question: given what this agent was asked to do and the identity it was given, how well did it perform?

The evaluator is itself an LLM agent. It receives the full context of the work: the task definition (title, description, deliverables), the agent’s identity (role and motivation), any artifacts the agent produced, log entries from execution, and timing data (when the task started and finished). From this, it scores four dimensions:

Dimension	Weight	What it measures
Correctness	40%	Does the output satisfy the task’s requirements and the role’s desired outcome?
Completeness	30%	Were all aspects of the task addressed? Are deliverables present?
Efficiency	15%	Was the work done without unnecessary steps, bloat, or wasted effort?
Style adherence	15%	Were project conventions followed? Were the motivation’s constraints respected?

The weights are deliberate. Correctness dominates because wrong output is worse than incomplete output. Completeness follows because partial work still has value. Efficiency and style adherence matter but are secondary—a correct, complete solution with poor style is more useful than an elegant, incomplete one.

The four dimension scores are combined into a single weighted score between 0.0 and 1.0. This score is the fundamental unit of evolutionary pressure.

Three-level propagation

A single evaluation does not merely update one record. It propagates to three levels:

1. **The agent’s performance record.** The score is appended to the agent’s evaluation history. The agent’s average score and task count update.
2. **The role’s performance record**—with the motivation’s ID recorded as `context_id`. This means the role’s record knows not just its average score, but *which motivation it was paired with* for each evaluation.
3. **The motivation’s performance record**—with the role’s ID recorded as `context_id`. Symmetrically, the motivation knows which role it was paired with.

This three-level, cross-referenced propagation creates the data structure that makes synergy analysis possible. A role's aggregate score tells you how it performs *in general*. The context IDs tell you how it performs *with specific motivations*. The distinction matters: a role might score 0.9 with one motivation and 0.5 with another. The aggregate alone would hide this.

What gets evaluated

Both done and failed tasks can be evaluated. This is intentional—there is useful signal in failure. Which agents fail on which kinds of tasks reveals mismatches between identity and work that evolution can address.

Human agents are tracked by the same evaluation machinery, but their evaluations are excluded from the evolution signal. The system does not attempt to “improve” humans. Human evaluation data exists for reporting and trend analysis, not for evolutionary pressure.

External evaluation sources

Not every signal about an agent's performance comes from an LLM reading its output. A trading agent might produce clean, well-structured code that scores 0.91 on internal evaluation—and lose money. A documentation agent might produce prose that the evaluator loves but that users find confusing. Internal quality assessment is necessary but not sufficient. The real test is what happens when the work meets the world.

Every evaluation carries a source field that identifies where the score came from. The internal auto-evaluator writes source: "llm". External evaluations use freeform tags that name their origin: "outcome:sharpe" for a portfolio's realized Sharpe ratio, "ci:test-suite" for a continuous integration result, "vx:peer-123" for a score received from a federated peer, "user:feedback" for a human's direct assessment. The tag is a string, not an enum—any external system can define its own source convention.

External evaluations enter the system through `wg evaluate record`:

```
wg evaluate record --task portfolio-q1 \  
  --source "outcome:sharpe" --score 0.72 \  
  --notes "Realized Sharpe below target"
```

The command requires a task in done or failed status, resolves the agent identity from the task's assignment, and writes the evaluation to the same store as internal evaluations. It propagates to the same three levels—agent, role with context, motivation with context. From the perspective of the performance records, an external evaluation is indistinguishable from an internal one except for the source tag.

This is where the evolutionary signal becomes rich. Consider an agent that scores 0.91 internally (clean code, complete deliverables, good style) but 0.72 on outcome (the code it wrote performed poorly in production). The evolver sees both scores in the performance summary. The gap between internal quality and external outcome is itself a signal—it suggests the role's desired outcome or the motivation's trade-offs need to account for domain-specific success criteria, not just code quality. The evolver can propose a mutation that sharpens the role toward outcomes the internal evaluator cannot see.

The five dimensions of external signal that can flow into a workgraph project—evaluation scores, new tasks, imported context, state changes, and event observations—form the system's interface with its environment. Evaluation is the most direct: it converts external reality into the same currency the evolver already reads.

Performance Records and Aggregation

Every role, motivation, and agent maintains a performance record: a task count, a running average score, and a list of evaluation references. Each reference carries the score, the task ID, a timestamp, and the crucial `context_id`—the ID of the paired entity.

From these records, two analytical tools emerge.

The synergy matrix

The synergy matrix is a cross-reference of every (role, motivation) pair that has been evaluated together. For each pair, it shows the average score and the number of evaluations. `wg agency stats` renders this automatically.

High-synergy pairs—those scoring 0.8 or above—represent effective identity combinations worth preserving and expanding. Low-synergy pairs—0.4 or below—represent mismatches. Under-explored combinations with too few evaluations are surfaced as hypotheses: try this pairing and see what happens.

The matrix is not a static report. It is a map of the agency's combinatorial identity space, updated with every evaluation. It tells you where your agency is strong, where it is weak, and where it has not yet looked.

Trend indicators

`wg agency stats` also computes directional trends. It splits each entity's recent evaluations into first and second halves and compares the averages. If the second half scores more than 0.03 higher, the trend is *improving*. More than 0.03 lower, *declining*. Within 0.03, *flat*.

Trends answer the question that aggregate scores cannot: is this entity getting better or worse over time? A role with a middling 0.65 average but an improving trend is a better evolution candidate than one with a static 0.70. Trends make the temporal dimension of performance visible.

Evolution

Evolution is the process of improving agency entities based on accumulated evaluation data. Where evaluation extracts signal from individual tasks, evolution acts on the aggregate—reading the full performance picture and proposing structural changes to roles and motivations.

Evolution is triggered manually by running `wg evolve`. This is a deliberate design choice. The system accumulates evaluation data automatically (via the coordinator's auto-evaluate feature), but the decision to act on that data belongs to the human. Evolution is powerful enough to reshape the agency's identity space. It should not run unattended.

The evolver agent

The evolver is itself an LLM agent. It receives a comprehensive performance summary: every role and motivation with their scores, dimension breakdowns, generation numbers, lineage, and the synergy matrix. It also receives strategy-specific guidance documents from `.workgraph/agency/evolver-skills/`—prose procedures for each type of evolutionary operation.

The evolver can have its own agency identity—a role and motivation that shape how it approaches improvement. A cautious evolver motivation that rejects aggressive changes will produce different proposals than an experimental one. The evolver's identity is configured in `config.toml` and injected into its prompt, just like any other agent. Two additional configuration options—`creator_agent` and `creator_model` (set via `wg config --creator-agent` and `wg config --creator-model`)—control the provenance metadata recorded on entities the evolver creates. When

set, newly created roles, motivations, and agents record these values, providing a traceable link between evolutionary output and the identity and model that produced it.

Strategies

Six strategies define the space of evolutionary operations:

Mutation. The most common operation. Take an existing role or motivation and modify it to address specific weaknesses. If a role scores poorly on completeness, the evolver might sharpen its desired outcome or add a skill reference that emphasizes thoroughness. The mutated entity receives a new content-hash ID—it is a new entity, linked to its parent by lineage.

Crossover. Combine traits from two high-performing entities into a new one. If two roles each excel on different dimensions, crossover attempts to produce a child that inherits the strengths of both. The new entity records both parents in its lineage.

Gap analysis. Create entirely new roles or motivations for capabilities the agency lacks. If tasks requiring a skill no agent possesses consistently fail or go unmatched, gap analysis proposes a new role to fill that space.

Retirement. Remove consistently poor-performing entities. This is pruning—clearing out identities that evaluation has shown to be ineffective. Retired entities are not deleted; they are renamed to `.yaml.retired` and preserved for audit.

Motivation tuning. Adjust the trade-offs on an existing motivation. Tighten a constraint that evaluations show is being violated. Relax one that is unnecessarily restrictive. This is a targeted form of mutation specific to the motivation's acceptable and unacceptable trade-off lists.

All. Use every strategy as appropriate. The evolver reads the full performance picture and proposes whatever mix of operations it deems most impactful. This is the default.

Each strategy can be selected individually via `wg evolve --strategy mutation` or combined as the default `all`. Strategy-specific guidance documents in the `evolver-skills` directory give the evolver detailed procedures for each approach.

Mechanics

When `wg evolve` runs, the following sequence executes:

1. All roles, motivations, and evaluations are loaded. Human-agent evaluations are filtered out—they would pollute the signal, since human performance does not reflect the effectiveness of a role-motivation prompt.
2. A performance summary is built: role-by-role and motivation-by-motivation scores, dimension averages, generation numbers, lineage, and the synergy matrix.
3. The evolver prompt is assembled: system instructions, the evolver's own identity (if configured), meta-agent assignments (so the evolver knows which entities serve coordination roles), the chosen strategy, budget constraints, retention heuristics (a prose policy from configuration), the performance summary, and strategy-specific skill documents.
4. The evolver agent runs and returns structured JSON: a list of operations (create, modify, or retire) with full entity definitions and rationales.
5. Operations are applied sequentially. Budget limits are enforced—if the evolver proposes more operations than the budget allows, only the first N are applied. After each operation, the local state is reloaded so subsequent operations can reference newly created entities.

6. A run report is saved to `.workgraph/agency/evolution_runs/` with the full transcript: what was proposed, what was applied, and why.

How modified entities are born

When the evolver proposes a `modify_role` operation, the system does not edit the existing role in place. It creates a *new* role with the modified fields, computes a fresh content-hash ID from the new content, and writes it as a new YAML file. The original role remains untouched.

The new role's lineage records its parent: the ID of the role it was derived from, a generation number one higher than the parent's, the evolver run ID as the creator, and a timestamp. For crossover operations, the lineage records multiple parents and takes the highest generation among them.

This is where content-hash IDs and immutability pay off. The original entity is a mathematical fact—its hash proves it has not been tampered with. The child is a new fact, with a provable link to its origin. You can walk the lineage chain from any entity back to its manually-created ancestor at generation zero.

Safety Guardrails

Evolution is powerful. The guardrails are proportional.

The last remaining role or motivation cannot be retired. The agency must always have at least one of each. This prevents an overzealous evolver from pruning the agency into nonexistence.

Retired entities are preserved, not deleted. The `.yaml.retired` suffix removes them from active duty but keeps them on disk for audit, rollback, or lineage inspection.

Dry run. `wg evolve --dry-run` renders the full evolver prompt and shows it without executing. You see exactly what the evolver would see. This is the first thing to run when experimenting with evolution.

Budget limits. `--budget N` caps the number of operations applied per run. Start small—two or three operations—review the results, iterate. The evolver may propose ten changes, but you decide how many land.

Self-mutation deferral. The evolver's own role and motivation are valid mutation targets—the system should be able to improve its own improvement mechanism. But self-modification without oversight is dangerous. When the evolver proposes a change to its own identity, the operation is not applied directly. Instead, a review meta-task is created in the workgraph with a `verify` field requiring human approval. The proposed operation is embedded in the task description as JSON. A human must inspect the change and apply it manually.

Lineage

Every role, motivation, and agent tracks its evolutionary history through a lineage record: parent IDs, generation number, creator identity, and timestamp.

Generation zero entities are the seeds—created by humans via `wg role add`, `wg motivation add`, or `wg agency init`. They have no parents. Their `created_by` field reads "human".

Generation one entities are the first children of evolution. A mutation from a generation-zero role produces a generation-one role with a single parent. A crossover of two generation-zero roles produces a generation-one role with two parents. Each subsequent evolution increments from the highest parent's generation.

The `created_by` field on evolved entities records the evolver run ID: "evolver-run-20260115-143022". Combined with the run reports saved in `evolution_runs/`, this creates a complete audit trail: you can trace any entity to the exact evolution run that created it, see what performance data the evolver was working from, and read the rationale for the change.

Lineage commands—`wg role lineage`, `wg motivation lineage`, `wg agent lineage`—walk the chain. Agent lineage is the most interesting: it shows not just the agent's own history but the lineage of its constituent role and motivation, revealing the full evolutionary tree that converged to produce that particular identity.

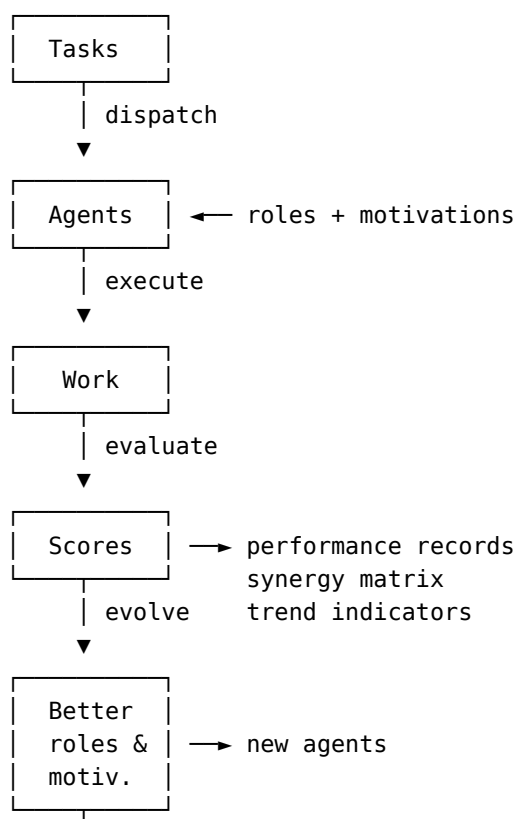
The Autopoietic Loop

Step back from the mechanics and see the shape of the whole.

Work enters the system as tasks. The coordinator dispatches agents—each carrying an identity composed of a role and a motivation—to execute those tasks. When a task completes, `auto-evaluate` creates an evaluation meta-task. The evaluator agent scores the work across four dimensions. Scores propagate to the agent, the role, and the motivation. Over time, performance records accumulate. Trends emerge. The synergy matrix fills in.

When the human decides enough signal has accumulated, `wg evolve` runs. The evolver reads the full performance picture and proposes changes. A role that consistently scores low on efficiency gets its description sharpened to emphasize economy. A motivation whose constraints are too tight gets its trade-offs relaxed. Two high-performing roles get crossed to produce a child that inherits both strengths. A consistently poor performer gets retired.

The changed entities—new roles, new motivations—are paired into new agents. These agents are dispatched to the next round of tasks. Their work is evaluated. Their evaluations feed the next evolution cycle.



└───→ back to dispatch

The meta-agents—the assigner that picks which agent gets which task, the evaluator that scores the work, the evolver that proposes changes—are themselves agency entities with roles and motivations. They too can be evaluated. They too can be evolved. The evolver can propose improvements to the evaluator’s role. It can propose improvements to *its own* role, subject to the self-mutation safety check that routes such proposals through human review.

This is what makes the system autopoietic: it does not just produce work, it produces the conditions for better work. It does not just execute, it reflects on execution and restructures itself in response. The identity space of the agency—the set of roles, motivations, and their pairings—is not static. It is a living population subject to selective pressure from the evaluation signal and evolutionary operations from the evolver.

Federation: cross-organizational learning

The autopoietic loop described above is closed within a single project. Federation opens it.

Agency entities—roles, motivations, agents, and their evaluation histories—can be shared across workgraph projects via `wg agency pull` and `wg agency push`. Named remotes point to other projects’ agency stores. When evaluations are transferred, they merge with local performance records: duplicates are identified by task ID and timestamp, and average scores are recalculated from the combined set. Content-hash IDs make this natural—an entity with the same identity-defining content has the same ID in every project, so deduplication is automatic.

What this means for evolution is concrete. A role that has been evaluated across three projects carries a richer performance record than one evaluated in a single project. The evolver sees a broader sample. A role that scores well on code tasks in one project but poorly on documentation tasks in another presents a clearer picture than either project could provide alone. Federation does not change the evolutionary mechanisms—it enriches the data they act on.

The sharing boundary is controlled by task visibility. Every task carries a `visibility` field: `internal` (the default—nothing crosses organizational boundaries), `public` (sanitized for open sharing—task structure without agent output or logs), or `peer` (richer detail for trusted peers—includes evaluations and workflow patterns). Trace exports (`wg trace export --visibility <zone>`) filter according to this field. The result is a structured, shareable view of work product—enough for a peer to learn from without exposing internal operational detail.

Functions: organizational routines

When a workflow pattern proves effective—a plan-implement-validate cycle that consistently produces high evaluation scores—it can be extracted into a reusable template. `wg func extract` reads the completed task graph, captures the task structure, dependencies, structural cycles, and agent role hints, and writes a parameterized function to `.workgraph/functions/`. `wg func apply` creates a fresh task graph from that template with new inputs.

These functions are the system’s organizational routines—the term Nelson and Winter (1982) used for the regular, predictable patterns of behavior that serve as an organization’s institutional memory. A routine extracted from a successful feature implementation captures not just what tasks to create, but what skills to require, what review loops to include, and what convergence patterns to expect. It is heritable (shareable across projects via the same YAML format), selectable (routines that produce good evaluation scores are retained; others are revised or abandoned), and mutable (a human or an LLM can edit the template to adapt it).

The connection to evolution is direct. Functions capture workflow structure. Evolution improves the agents that execute those workflows. Together, they represent two axes of organizational improvement: better processes and better performers. A well-extracted function dispatched to well-evolved agents is the system's equivalent of a mature team following a proven playbook.

But the human hand is always on the wheel. Evolution is a manual trigger, not an automatic process. The human decides when to evolve, reviews what the evolver proposes (especially via `--dry-run`), sets budget limits, and must personally approve any self-mutations. The system improves itself, but only with permission.

Practical Guidance

When to evolve. Wait until you have at least five to ten evaluations per role before running evolution. Fewer than that, and the evolver is working from noise rather than signal. `wg agency stats` shows evaluation counts and trends—use it to judge readiness.

Start with dry run. Always run `wg evolve --dry-run` first. Read the prompt. Understand what the evolver sees. This also serves as a diagnostic: if the performance summary looks thin, you need more evaluations before evolving.

Use budgets. `--budget 2` or `--budget 3` for early runs. Review each operation's rationale. As you build confidence in the evolver's judgment, you can increase the budget or omit it.

Targeted strategies. If you know what the problem is—roles scoring low on a specific dimension, motivations with constraints that are too strict—use a targeted strategy. `--strategy mutation` for improving existing entities. `--strategy motivation-tuning` for adjusting trade-offs. `--strategy gap-analysis` when tasks are going unmatched.

Seed, then evolve. `wg agency init` creates four starter roles and four starter motivations. These are generic seeds—competent but not specialized. Run them through a few task cycles, accumulate evaluations, then evolve. The starters are generation zero. Evolution produces generation one, two, and beyond—each generation shaped by the actual work your project requires.

Watch the synergy matrix. The matrix reveals which role-motivation pairings work well together and which do not. High-synergy pairs should be preserved. Low-synergy pairs are candidates for mutation or retirement. Under-explored combinations are experiments waiting to happen—assign them to tasks and see what the evaluations say.

Lineage as audit. When an agent produces unexpectedly good or bad work, trace its lineage. Which evolution run created its role? What performance data informed that mutation? The lineage chain, combined with evolution run reports, makes every identity decision traceable.

Mix internal and external signals. Do not evolve on internal evaluations alone if external outcome data is available. Record CI results, production metrics, or user feedback via `wg evaluate record --source <tag>`. The evolver is most effective when it sees both “the code was well-written” and “the code worked in practice”—the gap between the two is where the most useful mutations live.

Pull before evolving. If you maintain multiple workgraph projects or collaborate with peers, run `wg agency pull` before `wg evolve`. Federation imports evaluation data from remote stores, giving the evolver a broader performance picture. A role evaluated across three projects is a more reliable signal than one evaluated in one.

Extract routines from success. When a workflow pattern produces consistently high scores, extract it with `wg func extract`. The resulting function preserves the structure that worked. Combine this with evolution: evolve the agents, keep the proven process.